CSC1302 - Exam 3

Exam Schedule
Mon, Dec 8, 2025, 3:30pm-4:45pm
Classroom South 608

General Policies

- 20% of the total grade
- You can bring one A4-size (copy paper size) **handwritten** double-sided note for the exam. PRINTED NOTES WILL NOT BE ALLOWED.
- Write legibly; illegible answers will not be graded.
- Please bring your **Panther ID card** (or any other photo ID) for identity verification. Please show your ID when you submit your exam paper.
- Please arrive at least 5 minutes before the exam starts.
- Any form of academic dishonesty may result in your failure of this course and a recommendation to the Dean of Students for further action.
- Seats may be assigned randomly.

Syllabus

1. **Python Basics**

- One-way and two-way if statement
- For loop, nested-for loop
- Slicing a string/list using s[i:j]
- String operators and methods
- List operators and methods

2. **Classes**

- What are Classes and Objects? How to define a class in python and create its objects?
- What are instance attributes and instance methods? Examples? How to access them?
- What is the default constructor parameter? Examples?
- Class attribute vs instance attribute
- Object composition - demonstrate through an example.
- Demonstrate how memory deallocation/garbage collection works in Python.

3. **Inheritance**

- Inheritance - demonstrate with an example.
- Overriding class methods - examples.
- Understand 5 different types of inheritances.
- Diamond problem in Hybrid Inheritance & it’s solution.

4. **Recursion**
   - Discuss what is a recursive function with examples.
   - What is a 'base case' of a recursive function? Study at least 3 examples.
   - What is a recursive case of a recursive function? Study at least 3 examples.
   - Fibonacci sequence generation using recursion (with and without memoization)

5. **Analyzing Algorithms**

   - What is big-Oh notation and what is it used for?
   - What is time complexity? Space complexity?
   - Be able to simplify expressions according to big-Oh rules
   - Find out the big-Oh time and space complexity for a given code snippet.

6. Searching and Sorting Algorithms

- Big-Oh
  - Be able to provide examples of best/worst cases of the following algorithms:
    - Linear Search
    - Selection Sort
    - Insertion Sort
    - Quicksort
- Be able to implement binary search (both recursive and iterative)
- Be able to explain/implement/illustrate the sorting algorithms:
  - Selection sort
  - Insertion sort
  - Merge sort

7. Data Structures

- Stack, Queue
  - Understand common operations of stack and queue (push, pop, enqueue, dequeue etc.) and their time-complexity.
- Trees
  - Know tree terminology:
    - Root, Leaf
    - Child/Descendants, Parent/Ancestors
    - Subtree
    - Height
    - Balance
  - Be able to implement tree traversal algorithms (pre-order, in-order, and post-order).
  - Be able to write simple recursive tree functions.
- Graphs
  - Terminology
    - Directed/Undirected

        - Neighbors
    - Be able to convert graph visuals into adjacency list and adjacency matrix representations, and vice-versa.

8. Exceptions, Modules, File I/O, GUI

- Exceptions
    - Be able to identify the behavior of code snippets using try/except/finally
    - Be able to recognize common exception types (e.g. ZeroDivisionError, ValueError etc.)
    - Be able to define and raise custom exceptions.
- Modules
    - Be able to explain Python behavior around importing files.
        - import *, import foo as bar
        - from x import y ← still runs the entirety of x
    - Be able to explain what a module is and how it is helpful?
- File I/O
    - Be able to explain what the various open() modes are (e.g. ‘r’, ‘w’, ‘a’, ‘r+’)
        - Be able to choose the appropriate one for a given task.
    - Be able to recognize and explain the difference between relative and absolute paths.
    - Be able to use the core file API (read(), readlines(), write())
    - Write a Python function that reads a text file and prints the **word frequency**.
    - Write a Python function that reads a text file and prints the **unique words**.
    - Write a Python function that **searches** for a word/string within a text file and return it’s index (-1 if not found).

9. NumPy, Pandas, Matplotlib

- NumPy
    - Be able to describe what NumPy is and in what way it complements Python’s core abilities.
    - Be able to identify the shape of NumPy arrays.
    - Be able to recognize NumPy slicing/indexing behavior (e.g. arr[:, 1:])
    - Be able to recognize following NumPy methods (considering a 2-D array):
        - Deleting a row/column from the array
        - Inserting a row/column into the array
        - full(), zeros(), ones(), ravel(), reshape(), transpose() methods
        - Difference between row major order and column major order
    - Be able to apply arithmetic operators (+,-,*,/) and aggregate functions (mean/max/median) on 1D & 2D NumPy arrays.
- Pandas
    - What are Pandas Series and DataFrame? Differences?
    - Differences between a DataFrame and a 2D NumPy array?
    - Slicing a DataFrame using different slicing notations (e.g. data.iloc[1:3,5:]).
    - Purpose of ‘inplace’ parameter in pandas drop() method
    - How to delete rows and/or columns of a DataFrame containing missing values?

- Be able to demonstrate the result of 4 different types of joins between two given DataFrames.

- Matplotlib
  - Be able to write a program that displays a line/scatter plot for a given dataset.
  - How to add title, xlabel, ylabel, legend to a plot?
  - Purpose of nrows, ncols, and index parameter of the subplot() method?
  - Can you write a program that displays two/three different plots within a same figure?

[A few more topics will be added during the week of Dec 1st.]